Why is there no Community or Confluent Python support for timer-driven PTF UDFs?

Process Table Functions (PTFs) are currently JVM-only in Flink 2.1+ because the Python API lacks the runtime bindings required to support stateful, timer-driven table semantics. This is not a simple API gap—it reflects a deeper architectural limitation in PyFlink's execution model, which cannot provide the tight JVM integration required for native state access, timer services, and changelog-aware processing.

> **Note:** PTF support in PyFlink would require end-to-end runtime bindings (Py4J, proxy classes, state/timer abstractions) across the Python–JVM boundary—a non-trivial effort given their deep integration with Flink's native runtime.

In practice, this means any workload requiring advanced table-level state and timers must be implemented in Java, even in otherwise Python-based pipelines.

PTFs sit at the boundary of Flink's most advanced Table API capabilities, which are deeply tied to the JVM execution model.

Architecture Overview (Python ↔ JVM Boundary)

```
flowchart LR
    A[PyFlink Python API] -->|Py4J / Beam Fn API| B[Flink JVM Runtime]
    B --> C[State + Timers + Changelog]
    C --> D["PTF (Java Only)"]
```

Table of Contents

- **1.0 Why timers make the Python gap even wider**
- **2.0 Practical workarounds**
  - **2.1 — Option 1: Write the PTF in Java, call Python logic via subprocess/gRPC**
  - **2.2 — Option 2: Register a Java PTF from a PyFlink job**
  - **2.3 — Option 3: Approximate with PyFlink DataStream API**

1.0 Why timers make the Python gap even wider

The companion row-driven example explains the general reasons why PTFs are Java-only. Timer-driven PTFs add an additional layer of complexity:

Capability	Why it's hard in Python
Timer registration	`TimeContext.registerOnTime()` requires direct access to the JVM timer service. PyFlink's gRPC-based worker boundary adds round-trip latency that breaks the tight timing guarantees.
`onTimer()` callback	The JVM must call back into the Python worker when a timer fires. No such callback mechanism exists in PyFlink's current execution model.

Capability	Why it's hard in Python
Named timer replacement	Re-registering a named timer (the inactivity pattern) requires atomic replace semantics that would need to be serialized across the Python–JVM boundary.
`on_time` argument	The `DESCRIPTOR(event_time)` plumbing that connects SQL watermarks to the PTF's `TimeContext` has no Python-side equivalent.

These are *additional* to the state access, changelog, and set-semantic limitations described in the row-driven Python README.

2.0 Practical workarounds

The same workarounds from the row-driven example apply here, with one key note: **timer-based patterns are particularly well-suited to the DataStream API workaround** (Option 3), since PyFlink's `KeyedProcessFunction` does support timers.

2.1 — Option 1: Write the PTF in Java, call Python logic via subprocess/gRPC

Write the timer and state logic in Java (where PTFs are natively supported), and delegate any ML/AI inference to a Python service via gRPC or REST.

2.2 — Option 2: Register a Java PTF from a PyFlink job

```
t_env.execute_sql("""
    CREATE FUNCTION session_timeout_detector
      AS 'ptf.SessionTimeoutDetector'
      USING JAR '/path/to/timer-driven-ptf-udfs.jar'
""")

t_env.execute_sql("""
    CREATE FUNCTION per_event_follow_up
      AS 'ptf.PerEventFollowUp'
      USING JAR '/path/to/timer-driven-ptf-udfs.jar'
""")

t_env.execute_sql("""
    CREATE FUNCTION sla_monitor
      AS 'ptf.SlaMonitor'
      USING JAR '/path/to/timer-driven-ptf-udfs.jar'
""")
```

All three PTFs run in the JVM from the same JAR; your PyFlink job just references them in SQL.

2.3 — Option 3: Approximate with PyFlink DataStream API

PyFlink's `KeyedProcessFunction` supports both state and timers, making it the closest Python approximation to a timer-driven PTF:

```python
class SessionTimeoutDetector(KeyedProcessFunction):
    def open(self, runtime_context):
        self.event_count = runtime_context.get_state(
            ValueStateDescriptor("event_count", Types.LONG()))
        self.last_payload = runtime_context.get_state(
            ValueStateDescriptor("last_payload", Types.STRING()))

    def process_element(self, value, ctx):
        count = (self.event_count.value() or 0) + 1
        self.event_count.update(count)
        self.last_payload.update(value.payload)

        # Reset the inactivity timer
        ctx.timer_service().delete_event_time_timer(self._last_timer)
        self._last_timer = ctx.timestamp() + 300_000  # 5 minutes
        ctx.timer_service().register_event_time_timer(self._last_timer)

        yield Row(value.event_type, value.payload, count, False)

    def on_timer(self, timestamp, ctx):
        yield Row("session_timeout", self.last_payload.value(),
                  self.event_count.value(), True)
        self.event_count.clear()
        self.last_payload.clear()
```

Trade-off: This approach requires the DataStream API (not SQL), loses the declarative `PARTITION BY` syntax, and requires manual `ValueState` management. But it does give you timers in Python.